# Kubernetes Overview

**Kubernetes** (**K8s**) is an open-source container orchestration platform used to **automate** the **deployment**, **scaling**, and **management** of **containerized applications**.

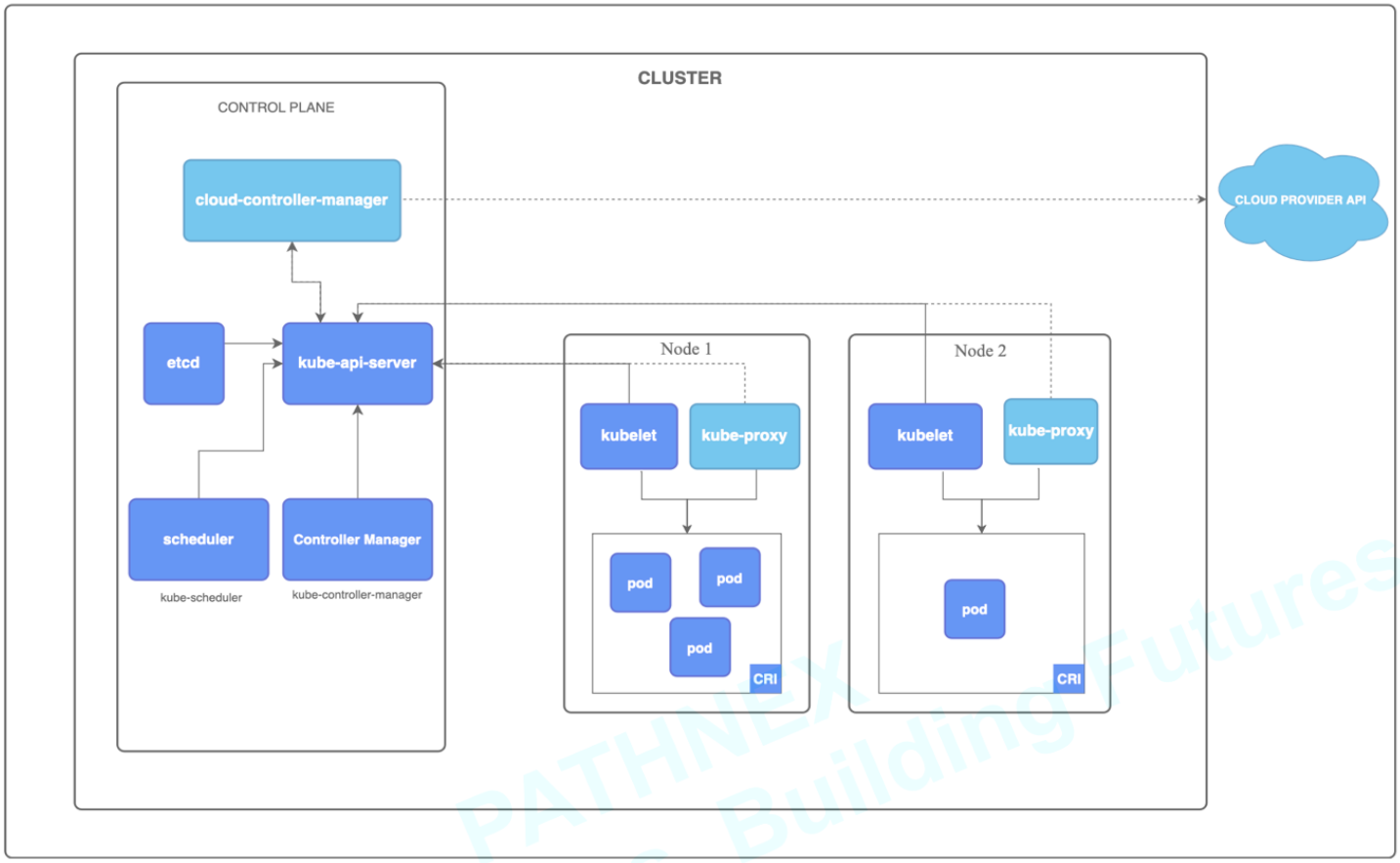
It helps **manage applications across a cluster of machines**, ensuring **high availability**, **scalability**, and **reliability**.

Kubernetes handles tasks such as:

- Deploying and managing containers (e.g., Docker)
- Scaling applications up or down
- Load balancing traffic
- Managing networking and storage
- Maintaining desired application state

In short: **Kubernetes simplifies managing containerized applications efficiently across multiple machines.**






Shaping Careers, Building Futures
PATHNEX

## Kubernetes Architecture and Components

The Kubernetes architecture consists of two main parts:

1. **Control Plane** (Master Node)
2. **Worker Nodes**

### 1. Control Plane (Master Node)

The Control Plane is responsible for the overall management of the Kubernetes cluster. It makes global decisions about the cluster (e.g., scheduling, deciding when and where to run pods, etc.) and maintains the cluster's desired state.

Key components of the Control Plane:

- **API Server (`kube-apiserver`)**:
  - The API server is the entry point to the Kubernetes cluster. It exposes the Kubernetes API and serves as the communication hub for all Kubernetes components (e.g., workers, etcd, controllers, etc.).

    - It processes REST requests, validates them, and updates the state of the system accordingly.
- **etcd**:
    - A distributed key-value store that holds all of the cluster's state data (configuration data, resource details, etc.). It is a crucial component for maintaining the desired state of the cluster, including all the cluster metadata.
    - **etcd** stores data about pods, deployments, services, and other resources.
- **Controller Manager (`kube-controller-manager`)**:
    - The controller manager ensures that the current state of the system matches the desired state. It continuously watches for changes and takes corrective actions.
    - Some of the controllers include:
        - **ReplicaSet Controller** (ensures the desired number of replicas for pods are maintained).
        - **Deployment Controller** (manages deployments).
        - **Job Controller** (handles batch jobs).
        - **Node Controller** (handles node lifecycle).
- **Scheduler (`kube-scheduler`)**:
    - The scheduler watches for newly created pods that have no assigned node and selects an appropriate worker node for them.
    - The scheduler considers resources (CPU, memory), affinity rules, and constraints to make placement decisions.
- **Cloud Controller Manager (`cloud-controller-manager`)**:
    - Manages cloud-specific components, such as load balancers and storage volumes, if your Kubernetes cluster is running in a cloud environment (AWS, GCP, Azure).
    - It abstracts cloud provider-specific logic out of the controller manager.

---

**2. Worker Nodes**

Worker nodes (also known as **Minions**) are where the actual application workloads (containers) run. Each node runs several important components that allow them to run the desired applications.

Key components of the Worker Nodes:

- **Kubelet**:
    - The **kubelet** is an agent that runs on each worker node. It ensures that containers are running in pods and are in the desired state (as defined by the control plane).
    - The kubelet communicates with the API server to ensure the nodes and pods are properly configured.
- **Kube Proxy**:
    - The **kube-proxy** is responsible for maintaining network rules on each node. It allows pods to communicate with each other and with services inside the cluster.

PATHNEX
Shaping Careers, Building Futures

    - It ensures that traffic is properly routed between pods and services, often by setting up iptables or IPVS rules.
- **Container Runtime**:
    - The container runtime is the software responsible for running containers. Kubernetes supports several container runtimes, such as Dockerd(Not being used after Kubernetes version 1.23), containerd, and CRI-O.
    - The runtime pulls container images, creates containers, and manages their lifecycle.
- **Pods**:
    - **Pods** are the smallest and simplest Kubernetes object. A pod is a logical host for one or more containers.
    - Pods in the same Kubernetes deployment can share networking and storage resources and are scheduled together onto the same worker node.
    - Each pod gets its own IP address, and the containers inside the pod share storage volumes.

---

## Other Key Kubernetes Concepts

- # Namespaces:

**Namespaces** are used to divide cluster resources into separate virtual environments within the same Kubernetes cluster.

They help in:

- Organizing resources
- Isolating applications
- Managing access and quotas
- Supporting multi-team or multi-project environments

PATHNEX
Shaping Careers, Building Futures

### Default Namespaces in Kubernetes

1. **default** – Used if no namespace is specified while creating resources.
2. **kube-system** – Contains system components like DNS, scheduler, controller manager, etc.
3. **kube-public** – Accessible to all users; mainly for public cluster information.
4. **kube-node-lease** – Used for node heartbeat and node availability tracking.

**In short:** Namespaces help organize and isolate resources inside a Kubernetes cluster.

- # Services:

A Service in Kubernetes defines a logical group of Pods and provides a stable way to access them. Since Pods are temporary and their IP addresses can change, a Service provides:

    - A stable IP address

- A DNS name
- Load balancing across multiple Pods

It uses labels and selectors to identify the required Pods and automatically routes traffic to available Pods, even if they are restarted or scaled.

In short: **A Service ensures reliable communication between different components in a Kubernetes cluster.**

# Service Types In Kubernetes

## 1. ClusterIP (Default)

- **Scope**: Internal to the cluster
- **Use case**: Microservices that need to communicate with each other within the cluster.
- **EKS behavior**: No external access. Only pods within the same cluster can reach this service.

---

## 2. NodePort

- **Scope**: External (limited)
- **Use case**: Basic external access by opening a static port on each worker node.
- **EKS behavior**:
  - EKS assigns a port from the `30000-32767` range on every worker node.
  - You access the service via `<NodeIP>:<NodePort>`.
  - Usually used for development or testing, not production.

---

## 3. LoadBalancer

- **Scope**: External (preferred for production)
- **Use case**: Expose services to the internet using AWS-managed Elastic Load Balancer (ELB).
- **EKS behavior**:
  - Creates an **AWS ELB (Classic, NLB, or ALB)** depending on configuration.
  - Traffic is routed to the backend pods.
  - Common for web apps or APIs needing public access.

---

PATHNEX
Shaping Careers, Building Futures

### 4. ExternalName

- **Scope**: Maps to external DNS
- **Use case**: Redirect a service name to an external DNS name.
- **EKS behavior**:
    - Doesn't create typical Kubernetes endpoints.
    - Used to integrate with external systems (like a database or legacy service outside Kubernetes).

---

### 5. Headless Service (special case of ClusterIP with `None`)

- **Scope**: Internal
- **Use case**: Service discovery, often used with StatefulSets (like Kafka, Cassandra).
- **EKS behavior**:
    - DNS returns a list of pod IPs instead of a single service IP.
    - Used for direct pod-to-pod communication.

PATHNEX
Shaping Careers, Building Futures

- # Deployments:

  A **Deployment** in Kubernetes manages a group of Pods and controls their lifecycle. It ensures the desired number of Pods are running at all times and automatically replaces failed Pods.

  A Deployment provides:

    - **Scaling** (increase or decrease number of Pods)
    - **Rolling updates** (update Pods without downtime)
    - **Rollback** (revert to a previous version if needed)

  **In short:** A Deployment helps manage, update, and maintain application Pods efficiently in a Kubernetes cluster.

- # ReplicaSets:

    - A **ReplicaSet** ensures that a specified number of pod replicas are running at any given time. It works alongside Deployments to manage the scaling and health of applications.

- **StatefulSets**:
  - A **StatefulSet** is used for managing stateful applications that require persistent storage, like databases. It ensures ordered deployment, scaling, and stable, unique network identifiers for each pod.

- **DaemonSets**:
  - A **DaemonSet** ensures that a particular pod is running on all (or specific) nodes in the cluster. It's commonly used for logging, monitoring, or networking agents.

- **Jobs and CronJobs**:
  - A **Job** ensures that a specified number of pods successfully terminate, useful for batch processing.
  - A **CronJob** runs jobs on a scheduled basis, similar to cron jobs in Unix/Linux.
- **Ingress**:
  - **Ingress** manages external access to services within the Kubernetes cluster, typically HTTP/HTTPS. It can also provide SSL termination, load balancing, and name-based virtual hosting.

PATHNEX Building Futures

```
Flow of Ingress (step-by-step)

Here's the real flow when a user hits your app:

1. User hits domain (pathnex.com)
   ↓
2. DNS resolves to external IP
   ↓
3. Cloud Load Balancer (Service type=LoadBalancer)
   ↓
4. Ingress Controller (NGINX / Traefik pod)
   ↓
5. Ingress Resource (rules)
   ↓
6. Kubernetes Service
   ↓
7. Pod (your application)
```

# Real Time Production setup

## There are 2 ways to deploy application

## ✅ 1. Standard Production Flow (NGINX Ingress)

This is the most common setup:

```
User → pathnex.com
   ↓
DNS → points to external Load Balancer IP
   ↓
Cloud Load Balancer (Service type=LoadBalancer)
   ↓
NGINX Ingress Controller (pods)
   ↓
Ingress Resource (routing rules)
   ↓
Kubernetes Service (type=ClusterIP)
   ↓
Pod (your app)
```

### 🔑 Key points

- **LoadBalancer** = public entry point
- **NGINX Ingress** = HTTP routing (host/path)
- **ClusterIP** = internal service communication
- ❗ No need to think about NodePort here—it's abstracted away

## ✅ 2. Service Mesh Flow (Istio)

If you're using Istio, the ingress changes:

```
User → pathnex.com
   ↓
DNS
   ↓
Cloud Load Balancer
   ↓
Istio Ingress Gateway
   ↓
VirtualService (routing rules)
   ↓
Kubernetes Service (ClusterIP)
   ↓
Pod (+ sidecar proxy)
```

### 🔑 Key points

- **Istio Gateway** replaces NGINX Ingress
- **VirtualService** replaces Ingress rules
- Each pod has a **sidecar proxy (Envoy)**
- More powerful (traffic splitting, retries, observability)

### ⚖️ NGINX vs Istio (simple view)

| Feature | NGINX Ingress | Istio |
|---|---|---|
| Simplicity | ✅ Easy | ❌ Complex |
| Routing | ✅ Basic | ✅ Advanced |
| Traffic control | ❌ Limited | ✅ Powerful |
| Overhead | ✅ Low | ❌ Higher |

---

# Kubernetes Networking

- **Pod-to-Pod Networking**:
  - Pods in a Kubernetes cluster can communicate with each other over a flat network. Each pod is assigned a unique IP address within the cluster.
- **Service Discovery**:
  - Kubernetes automatically assigns a DNS name to services, allowing pods to easily discover and access each other by name.
- **Network Policies**:
  - Kubernetes provides **Network Policies** to control the communication between pods based on labels and namespaces, enhancing security.

**EKS Pricing Model:** https://aws.amazon.com/eks/pricing/

## What is Affinity?

Affinity is about telling Kubernetes:

"I want my pod to run **near something** (or avoid something)."

It helps control **where pods are scheduled** based on rules.

There are **two main kinds**:

---

### 1️⃣ Node Affinity

- Tells your pod:
  👉 "Only run on certain nodes."
- Used when you want pods on specific hardware or environments

### ✅ Examples:

- "Run on nodes with **SSD disks**"
- "Run only in **us-east-1 zone**"
- "Run on **GPU-enabled nodes**"

PATHNEX
Shaping Careers, Building Futures

### 🧠 Example YAML:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disk
          operator: In
          values:
          - ssd
```

---

## 2️⃣ Pod Affinity / Anti-Affinity

This controls how pods are placed **relative to other pods**.

---

### ✅ Pod Affinity (bring pods together)

- Tells Kubernetes:
  👉 "Place this pod **close to other pods**"

Example:

- "Run backend pods near frontend pods"
- "Keep related services in the same zone"

---

### ❌ Pod Anti-Affinity (spread pods apart)

- Tells Kubernetes:
  👉 "Do NOT place this pod near other pods"

Example:

- "Don't put two replicas on the same node"
- "Spread pods across nodes for high availability"

---

### 🧠 Example YAML (Pod Anti-Affinity)

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values:
          - my-app
      topologyKey: "kubernetes.io/hostname"
```

PATHNEX
Shaping Careers, Building Futures

---

## ⚡ Important Concepts

### 🔶 Required vs Preferred

- **requiredDuringSchedulingIgnoredDuringExecution**
  - Hard rule (must follow)
- **preferredDuringSchedulingIgnoredDuringExecution**
  - Soft rule (try best)

---

### 🎯 Simple Mental Model

```
Node Affinity        → Choose WHICH node
Pod Affinity         → Choose WHICH pods to be near
Pod Anti-Affinity    → Choose WHICH pods to avoid
```

---

## 🔄 Pod Affinity vs Node Affinity (super short)

| Type | Meaning | Based On |
|---|---|---|
| Node Affinity | Run on specific kinds of **nodes** | Node labels |
| Pod Affinity | Run near specific **pods** | Pod labels |
| Pod AntiAffinity | Avoid being near specific **pods** | Pod labels |

---

## 🧠 Real-Life Example

Imagine you're deploying 3 copies of your app:

- 🟢 With **Pod AntiAffinity**, you tell Kubernetes:
  *"Spread them out so they're not all on the same machine."*
  (Good for high availability)
- 🟢 With **Node Affinity**, you say:
  *"Only use nodes with `type=fast`."*
  (Good for performance)

## 😷 What is a Taint?

A **taint** is like putting a sign on a **node** that says:

❌ *"Don't come here unless you can handle this."*

It prevents **most pods** from being scheduled on that node **unless they "tolerate" the taint**.

PATHNEX Shaping Careers, Building Futures

# 🛡️ What is a Toleration?

A **toleration** is something you put on a **pod** that says:

✅ *"It's okay, I'm allowed on that tainted node."*

So, **taints are on nodes**, and **tolerations are on pods**.

---

# 🧠 Real-Life Example

Let's say you have a node with **very expensive GPU** hardware, and you want only special pods to run there.

- You add a **taint** on that node:

```
bash

kubectl taint nodes gpu-node key=gpu:NoSchedule
```

- That means: "No pod is allowed here unless it tolerates `key=gpu`."
- Now, on your special GPU pod, you add a **toleration**:

```
yaml

tolerations:
  - key: "gpu"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
```

This pod says: "I can tolerate the `gpu` taint," and it gets scheduled on the GPU node.

---

# 🧾 Common Effects of Taints

| Effect | Meaning |
|---|---|
| `NoSchedule` | Don't schedule pods here **unless they tolerate it** |
| `PreferNoSchedule` | Try to avoid scheduling here, but not strictly |
| `NoExecute` | Kick out existing pods that don't tolerate it (and block new ones too) |

---

## 🔁 Summary

| Thing | Where? | What it does |
|---|---|---|
| **Taint** | Node | Keeps most pods **off** the node |
| **Toleration** | Pod | Says: *"I'm okay to run on that tainted node."* |

PATHLEX
Shaping Careers, Building Futures

# 🔷 Probes in Kubernetes (EKS)?

In Kubernetes (including Amazon EKS), **probes are health checks** for your application containers.

Think of them like:

“Is my app alive?”
“Is it ready to serve users?”
“Should I restart it?”

---

## 🔷 There are 3 Types of Probes

### 1. 🟢 Liveness Probe (Is it alive?)

- Checks if your app is **still running properly**
- If it fails → Kubernetes **restarts the container**

👉 example:

Like checking if a machine is completely stuck. If yes, reboot it.

---

### 2. 🔵 Readiness Probe (Can it take traffic?)

- Checks if your app is **ready to handle requests**
- If it fails → traffic is **stopped**, but container is NOT restarted

👉 example:

A restaurant is open but not ready to serve food yet → don’t send customers in.

---

### 3. 🟡 Startup Probe (Has it started yet?)

- Used for slow-starting apps
- Prevents liveness probe from killing the app too early

👉 example:

Don’t panic-restart a computer while it’s still booting up.

---

PATHNEX
Shaping Careers, Building Futures

## 🔷 Why Probes Matter in EKS

Without probes:

- Your app might **crash silently**
- Users may hit **broken services**
- Kubernetes won't know when to fix things

With probes:

- Auto-restarts broken apps
- Only sends traffic to healthy pods
- Improves reliability

---

## 🔷 Simple YAML Example

```
containers:
  - name: my-app
    image: my-app-image

    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 5

    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
```

PATHNEX
Shaping Careers, Building Futures

---

## 🔷 Easy Way to Remember

| Probe Type | Question it asks | Action taken |
|---|---|---|
| Liveness | "Is it broken?" | Restart container |
| Readiness | "Can it serve users?" | Stop traffic |
| Startup | "Has it started yet?" | Wait before checks |

---

## 🔷 Real-World Analogy

Think of a **food delivery app**:

- Startup probe → App is launching 📱
- Readiness probe → App is ready to take orders 🍔
- Liveness probe → App froze → restart it 🔄